

3-1 Characters

C++ has the letters and digits, as show below:

Uppercase: A, B, C, ... , Z

Lowercase: a, b, c, ... , z

Digits: 0, 1, 2, ... ,9

Special Characters: All characters other than listed treated as special characters for example:

+	-	*	/	Λ
(	[	{	}	]
)	<	=	>	,
“	.	:	;	␣
(Double quotation)	(Dot)	(Colon)	(Semicolon)	(Blank Space)

✚ In C++ language, upper case and lower case letters are distinct and hence there are 52 letters in all. Each letter has a distinct ASCII value that the computer uses to differentiate between the cases. For example, 'A' has an ASCII (American Standard Code for Information Interchange.) value of 65, and 'a' has an ASCII value of 97.

✚ For example, **bag** is different from **Bag** which is different from **BAG**.

3-2 Identifier

An **identifier** is a name given to some program entity, such as variable, constant, array, function, structure, or class. An identifier is a sequence of alphanumeric (alphabetic and numeric) characters, the first of which must be a letter, and can't contain spaces. In C++, the maximum length of an identifier is not fixed by the language standard as machine

dependent but is **implementation-defined**, meaning it depends on the specific **compiler** and **linker** being used. C++ allows identifiers of up to **127 characters**.

A **variable** should not begin with a digit. C++ does not set a maximum length for an identifier. Some examples of valid identifiers are as follows:

The name of variable	The length
My_name	7 char.
B	1 char.
i	1 char.

Rules for Invalid Identifiers

- 1- **Starts with a digit.**
- 2- **Contains prohibited characters:** Only letters, digits, and the underscore () are allowed. Special characters like spaces, hyphens (-), !, @, #, %, etc., are not permitted.
- 3- **Is a reserved keyword:** You cannot use a C++ reserved word (like `int`, `float`, `return`, `while`, `class`, etc.) as an identifier because it has a predefined meaning.
- 4- **Contains a double underscore or starts with an underscore followed by an uppercase letter** (for implementation names): Identifiers containing `__` (double underscore) or beginning with `_` followed by an uppercase letter are generally reserved for the C++ implementation itself and should be avoided in user code to prevent conflicts.

By following these rules, you can ensure your variable, function, and class names are valid and compile correctly.

Examples of **invalid** identifiers are:

Invalid variable	The reason
3ab	Started with digit
a()test	Contained special char.
red rose	Contained space
int	Reserved word
First-name	Contains a hyphen
@username	Starts with a special character

3-3 Keywords

The keywords are also identifiers but cannot be user defined, since they are reserved words. They have special, predefined meanings that the compiler understands and therefore cannot be used as variable or function names. All the keywords should be in lower case letters. Reserved words cannot be used as variable names or constant.

The following are some **Categories of C++ Keywords**

Category	Keywords
Data Types	bool, int, long, short, float, double, void
Control Flow	if, else, switch, case, for, while, do, break, continue
Boolean & Null	true, false, nullptr
Memory Management	new, delete, sizeof, alignas, alignof

3-4 Constants

There are three types of constants: **string constants**, **numeric constants**, and **character constants**.

1. **String Constants:** A string constants are a sequence of alphanumeric characters enclosed in double quotation marks whose maximum length is 255 characters. In the following are examples of valid string constants: ("The result=", "RS 2000.00", "This is test program"). The invalid string constants are like: (Race, "My name, 'this'").
2. **Numeric Constants:** Numeric constants are positive or negative numbers. There are four types of numeric constants: integer, floating point, hexadecimal, and octal.
3. **Character Constants:** A character represented within single quotes denotes a character constant, for example 'A', 'a', ':', '?', etc.

3-5 Operators

C++ operators are the symbols that operate on values to perform specific mathematical or logical computations on given values. They are the foundation of any programming language.

- 1- **Arithmetic operators:** These operators require two variables to be evaluated:

+ addition	- subtraction	* multiplication
/ division	% modulo (remainder of an integer division)	

In C++ the division results are:

Integer/ integer= integer 39/7=5

Integer/ float = float 39 /7 .0 =5.57

float / integer = float 39.0/7 =5.57

float / float = float 39.0/7.0=5.57

Arithmetic operators as per precedence:

- () for grouping the variables.
- Unary for negative number.
- * / multiplication & division.
- + - addition and subtraction.

Example: find the result of the following equation :

$X+Y*X-Z$ where $X=5$, $Y=6$, and $Z=8$.

$5 + (6*5)-8$

$(5+30)-8$

$35-8= 27$

2- **Assignment Operators:** The operational assignment operator has the form:

$x=x+5;$

Variable= variable operator expression;

$y=y*10;$

The operational assignment operator can be written in the following form:

$x+=5;$

Variable operator = expression

Example:

$x+=5;$

Variable operator = expression

$y*= 10;$

It is used to assign back to a variable, a modified value of the present holding:

=	Assign right hand side (RHS) value to the left hand side (LHS).
+=	Value of LHS var. will be added to the value of RHS and assign it back to the var. in LHS.
-=	Value of RHS var. will be subtracted to the value of LHS and assign it back to the var. in LHS.
*=	Value of LHS var. will be multiplied to the value of RHS and assign it back to the var. in LHS.
/=	Value of LHS var. will be divided to the value of RHS and assign it back to the var. in LHS.
%=	The remainder will be stored back to the LHS after integer division is carried out between the LHS var. and the RHS var.
>>=	Right shift and assign to the LHS.
<<=	Left shift and assign to the LHS.
&=	Bitwise AND operation and assign to LHS
 =	Bitwise OR operation and assign to LHS
~=	Bitwise complement operation and assign to LHS

Exercise:

Rewrite the equivalent statements for the following examples, and find it results.

Assume: X=2, Y=3, Z=4, V=12, C=8.

Example	Equivalent statement	Result
X+=5	X=X+5	X=7
Y-=8	Y=Y-8	Y= -5
Z*=5	Z=Z*5	Z= ???
V/=4	???	V= ???
C%=3	???	C= ???

- 3- **Comparison and logical operators:** It has three types relational operators, equality operators, and logical operators.

(a) Relational operators: < less than, > greater than, <= less than or equal, >= greater than or equal, an expression that use relational operators return the value of one if the relational is TRUE and zero otherwise.

Ex:

3 > 4false,

6 <=2false

10>-32.....true

(23*7)>= (-67+89)..... true

(b) Equality operators: == equal to, != not equal to

Ex:

a=4, b=6, c=8.

A==b.....false

(a*b) !=c.....true

's' == 'y'false.

(c) Logical operators: The logical expression is constructed from relational expressions by the use of the logical operators **not(!)**, **and(&&)**, **or(||)**.

AND (&&) Table:		
A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

AND (&&) Table:		
A	B	A && B
1	1	1
1	0	0
0	1	0
0	0	0

OR () Table:		
A	B	A B
T	T	T
T	F	T
F	T	T
F	F	F

NOT (!) Table:	
A	!A
T	F
F	T

OR () Table:		
A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

NOT (!) Table:	
A	!A
1	0
0	1

Example:

Let a=4, b=5, c=6 then the following expression can be constructed:

(a<b)&&(b<c)	(a<b) (b>c)	!(a<b) (c>b)	(a<b) (b>c)&&(a>b) (a>c)
T && T	T F	!(T) T	T F && F F
T	T	F T	T F F
		T	T F
			T

Example: assume: X=0, Y=1, Z=1. Find the following expression:

$$M = ++X \parallel ++Y \&\& ++Z$$

$$M = ++X \parallel ++y \&\& ++z$$

$$= 1 \parallel (2 \&\& 2)$$

$$= T \parallel (T \&\& T)$$

$$= T \parallel T$$

$$= T$$

$$= 1$$